# *Exercise 1*
# *Fun with Least Squares Optimization*

### How to approach the tasks on this exercise sheet?

Work in teams! Why? Because it hones your interpersonal communication skills! Those will always be important, even in the era of ever stronger AI.

Be ambitious and try to solve the following problems yourself. If you work with AI assistance, you don't have to code everything yourself. But try to figure out what needs to be coded so that you can provide specific instructions for your AI companion.

If your main goal is to learn to cooperate with a capable AI companion, then consider the following questions for your interactions:

- `How could we approach this task ?`
- `Why should we consider this approach ?`
- `Are there any other or better approaches ?`

Once you have solved a task together with an AI, you might want to ask these questions:

- `What are the main takeaways from this task ?`
- `Why may our instructor may have set this task ?`
- `What could he have wanted us to learn by solving it ?`

## Contents

Problem 1.1: beware of numerical instabilities . . . . . 2
Problem 1.2: learning cellular automaton updates . . . . . 5
Problem 1.3: estimating fractal dimensions in images . . . . . 12
Problem 1.4: fitting exponential functions to data . . . . . 15
Problem 1.5: working with barycentric coordinates . . . . . 17
References . . . . . 20

# Problem 1.1: beware of numerical instabilities



**Note**
Every student in every team should do the following tasks on their own because everybody needs to know about potential pitfalls when using digital computers to obtain numerical solutions to computational problems!

The phenomenon you are to explore here is so important to know about that we shall guide you through your tasks. But don't take them lightly! They reveal that mathematical theory and digital computing practice are not always well aligned. If you want to develop machine learning solutions for real world applications must be aware of this …

To BEGIN WITH, consider the following basic or so called *ordinary* least squares problem

$$\boldsymbol{w}_\star = \underset{\boldsymbol{w}}{\operatorname{argmin}} \left\| \boldsymbol{X}^\mathsf{T}\boldsymbol{w} - \boldsymbol{y} \right\|^2 \tag{1}$$

and let the *design matrix* $\boldsymbol{X}^\mathsf{T}$ and *target vector* $\boldsymbol{y}$ be

$$\begin{aligned} \boldsymbol{X}^\mathsf{T} &= \begin{bmatrix} 1.00000 & -1.00000 \\ 0.00000 & 0.00001 \\ 0.00000 & 0.00000 \end{bmatrix} \\ \boldsymbol{y} &= \begin{bmatrix} 0.00000 \\ 0.00001 \\ 0.00000 \end{bmatrix} . \end{aligned} \tag{2}$$

Next, observe that this specific instance of an LSQ problem is perfectly solved by vector

$$\boldsymbol{w}_\star = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \tag{3}$$

because $\boldsymbol{X}^\mathsf{T}\boldsymbol{w}_\star = \boldsymbol{y}$ so that $\left\| \boldsymbol{X}^\mathsf{T}\boldsymbol{w}_\star - \boldsymbol{y} \right\|^2 = 0$ which is the smallest possible value our quadratic objective function can assume.

SOLVING THE PROBLEM in (1) with the ingredients in (2) is easy because the latter are small enough for us to "see" the solution. In practice, problems rarely are this simple. However, we already know that we can solve ordinary LSQ problems by computing

$$\boldsymbol{w}_\star = \left[\boldsymbol{X}\boldsymbol{X}^\mathsf{T}\right]^{-1}\boldsymbol{X}\boldsymbol{y} \, . \tag{4}$$

In other words, we know how to solve LSQ problems for any $\boldsymbol{X}^\mathsf{T}$ and $\boldsymbol{y}$ (as long as they meet certain algebraic conditions ($\boldsymbol{X}^\mathsf{T}$ must not be rank-deficient, $\boldsymbol{X}^\mathsf{T}$ and $\boldsymbol{y}$ must be of commensurable dimensions, etc.)).

SO THEN WHAT HAPPENS if we implement the algebraic solution in (4) to let a computer solve our exemplary LSQ problem?

## Task 1.1.1

Run the following `Python` / `numpy` code snippet and marvel at the output it produces

```
import numpy as np
import numpy.linalg as la

matX = np.array([[ 1.00000, 0.00000, 0.00000],
                 [-1.00000, 0.00001, 0.00000]])
vecY = np.array( [ 0.00000, 0.00001, 0.00000] )

vecW = la.inv(matX @ matX.T) @ matX @ vecY

print ('pen-and-paper solution:', vecW)
```

Below, you will find further tasks. Yet, to make sense of what is coming, we first need some background information ...

## Background Info

A matrix is said to be "tall" or "thin" if it has more rows than columns. Matrix $\boldsymbol{X}^\mathsf{T} \in \mathbb{R}^{r\times c}$ in (2) is such a matrix and therefore has a **QR decomposition**

$$\boldsymbol{X}^\mathsf{T} = \boldsymbol{Q}\boldsymbol{R} \tag{5}$$

such that

$$\begin{aligned} &\boldsymbol{Q} \in \mathbb{R}^{r\times r} \text{ is } \textbf{orthogonal} \\ &\boldsymbol{R} \in \mathbb{R}^{r\times c} \text{ is } \textbf{upper triangular} \,. \end{aligned} \tag{6}$$

Since the $r-c$ bottom rows of an upper triangular matrix contain only 0s, we may partition the product $\boldsymbol{Q}\boldsymbol{R}$ as follows

$$\boldsymbol{Q}\boldsymbol{R} = [\boldsymbol{Q}_1 \;\; \boldsymbol{Q}_2] \begin{bmatrix} \boldsymbol{R}_1 \\ \boldsymbol{0} \end{bmatrix} = \boldsymbol{Q}_1\boldsymbol{R}_1 \tag{7}$$

where the $c$ different columns of $\boldsymbol{Q}_1 \in \mathbb{R}^{r\times c}$ are still orthogonal and $\boldsymbol{R}_1 \in \mathbb{R}^{c\times c}$ is still upper triangular.



**Info**
`numpy`'s `linalg` module provides a function `qr()` whose default behavior is to return $\boldsymbol{Q}_1$ and $\boldsymbol{R}_1$ rather than $\boldsymbol{Q}$ and $\boldsymbol{R}$.

Before turning the page, it might be a good idea to convince yourself that all statements so far are true and make sense. Also recall that, for any orthogonal matrix $\boldsymbol{O}$, we have the equalities $\boldsymbol{O}^\mathsf{T}\boldsymbol{O} = \boldsymbol{O}\boldsymbol{O}^\mathsf{T} = \boldsymbol{I}$.

Now, OBSERVE the following: If $\boldsymbol{X}^\mathsf{T} = \boldsymbol{Q}_1 \boldsymbol{R}_1$ and the columns of $\boldsymbol{Q}_1$ are orthogonal, then

$$\begin{aligned} \boldsymbol{w}_\star = \left[\boldsymbol{X}\boldsymbol{X}^\mathsf{T}\right]^{-1}\boldsymbol{X}\boldsymbol{y} &= \left[\boldsymbol{R}_1^\mathsf{T}\boldsymbol{Q}_1^\mathsf{T}\boldsymbol{Q}_1\boldsymbol{R}_1\right]^{-1}\boldsymbol{R}_1^\mathsf{T}\boldsymbol{Q}_1^\mathsf{T}\boldsymbol{y} \\ &= \left[\boldsymbol{R}_1^\mathsf{T}\boldsymbol{R}_1\right]^{-1}\boldsymbol{R}_1^\mathsf{T}\boldsymbol{Q}_1^\mathsf{T}\boldsymbol{y} \\ &= \boldsymbol{R}_1^{-1}\boldsymbol{R}_1^{-\mathsf{T}}\boldsymbol{R}_1^\mathsf{T}\boldsymbol{Q}_1^\mathsf{T}\boldsymbol{y} \\ &= \boldsymbol{R}_1^{-1}\boldsymbol{Q}_1^\mathsf{T}\boldsymbol{y}\,. \end{aligned} \tag{8}$$

WHAT WE JUST FOUND is that $\boldsymbol{w}_\star = \boldsymbol{R}_1^{-1}\boldsymbol{Q}_1^\mathsf{T}\boldsymbol{y}$ for which we note that the inverse of an upper triangular matrix (if it exists) can be computed quickly and with high numerical robustness …

## Task 1.1.2

Still consider the problem in (1) with the ingredients in (2) and implement `numpy` code to compute $\boldsymbol{w}_\star = \boldsymbol{R}_1^{-1}\boldsymbol{Q}_1^\mathsf{T}\boldsymbol{y}$. Look at the result you get.

## Task 1.1.3

Still consider the problem in (1) with the ingredients in (2) and implement `numpy` code that uses the `linalg` function `lstsq()` to compute $\boldsymbol{w}_\star$. Look at the result you get. What does it suggest: Does the special purpose function `lstsq()` solve least squares problems via equation (4) or not?

## Task 1.1.4

Think about the meaning behind all this. Here are a few questions to guide your thoughts:

- Are real numbers and their floating point representations on a digital computer the same thing?
- What does it mean to speak about machine precision w.r.t. floating point arithmetic?
- What do we have to keep in mind when working with either very large or very small floating point numbers?
- What do you find when you use pen and paper to compute the entries of $\boldsymbol{X}\boldsymbol{X}^\mathsf{T}$?
- What do you find when you use your computer to compute the entries of $\boldsymbol{X}\boldsymbol{X}^\mathsf{T}$?
- Can you blindly trust any piece of numerical software that some dude from somewhere has uploaded to github or even to huggingface?

# Problem 1.2: learning cellular automaton updates

In our lectures, we already encountered the idea of *feature maps* which transform lower dimensional data to higher dimensional representations. Here, we explore this idea in a somewhat unusual context …

## Background Info

A one-dimensional or *elemental* **cellular automaton** consists of two basic components, namely

1. an *infinite sequence of cells* $x_j$ with $j \in \mathbb{Z}$; at any discrete point in time $t \in \mathbb{N}$, each cell is in either one of two states which are called 0 (`off`) or 1 (`on`) so that we have

   $$x_j[t] \in \{0, 1\} \tag{9}$$

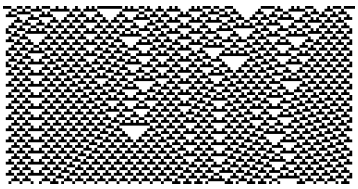
2. an *update rule* $f : \{0, 1\}^3 \to \{0, 1\}$ for how to update a cell based on its own and its two neighbors' current states; for a cell's state transitions from time $t$ to time $t + 1$, we therefore have

   $$x_j[t + 1] = f\big(x_{j-1}[t], x_j[t], x_{j+1}[t]\big) \ . \tag{10}$$

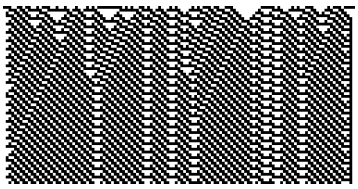
At time $t = 0$, the $x_j$ are (randomly) initialized, and, at any subsequent time $t$, all cells are updated simultaneously. Here is an illustrative example where we plot 0 as □ and 1 as ■

$t$ ⋯ □□□□□□□□□□□□□□□■□□□□□□□□□□□□□□□□□⋯
$t+1$ ⋯ □□□□□□□□□□□□□□□■□■□□□□□□□□□□□□□□□⋯

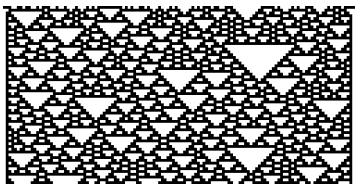
This update process continues forever and it is common to plot subsequent (finitely sized) state sequences below one another to visualize the evolution of an automaton under a certain rule. Here are some examples of possible evolutions all starting from the same initial configuration but under different update rules



rule 108



rule 110



rule 126

If you are wondering about the names of these exemplary rules, we'll say more about them soon, but first ...

Henceforth, we will reduce notational clutter. That is, instead of writing the somewhat tedious expression in (10), we henceforth simply write

$$y = f(\boldsymbol{x}) \tag{11}$$

where $\boldsymbol{x} \in \{0,1\}^3$ and $y \in \{0,1\}$.

The naming convention for the rules which govern the evolution of elemental cellular automata is due to **Stephen Wolfram** who studied them extensively [1]. To get to grips with the rationale behind his convention, consider the following tabular representations of update rules 110 and 126 where we suggestively label the columns of both tables:

| $\boldsymbol{X}^\mathsf{T}$ | | | $\boldsymbol{y}$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |

rule 110

| $\boldsymbol{X}^\mathsf{T}$ | | | $\boldsymbol{y}$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |

rule 126

Looking at this tables, we observe: For $\boldsymbol{x} \in \{0,1\}^3$, there are $2^3 = 8$ possible inputs for an update rule. For each input, there are 2 possible outputs. Hence, the total number of possible rules is $2^{2^3} = 256$.

We may collect the possible input patterns in an $8 \times 3$ design matrix $\boldsymbol{X}^\mathsf{T}$ and the outputs of each rule in an 8 dimensional target vector $\boldsymbol{y}$. Converting a rule's binary target vector (read from bottom to top) into a decimal number then gives the rule's name.

## Task 1.2.1

Implement `Python` / `numpy` code that produces pictures as shown on the previous page. Make sure your code can handle any rule $f_r$ for $r \in \{0, 1, \ldots, 255\}$.

Of course, we cannot practically work with infinite sequences of cells. In your implementation, you would therefore have to consider only a finite number of cells, say $n_c = 127$. By the same token, we cannot practically realize an update process that runs forever. It is

thus suggested that you only consider a finite number of rows, say $n_r = 64$ when plotting the evolution of an automaton.

Randomly initialize the individual cells of your automaton to 0 or 1 and then run it using rules 60 and 102. Produce pictures of your results.

## More Background Info

Now, consider a seemingly crazy idea, namely the following substitution for the states of a cell of a cellular automaton

$$\begin{array}{rcl} 0 & \mapsto & +1 \\ 1 & \mapsto & -1\ . \end{array} \tag{12}$$

People like this representation of binary states because there is a simple mapping from the ordered set $\{0, 1\}$ to the ordered set $\{+1, -1\}$, namely

$$x \in \{0, 1\} \quad \mapsto \quad x' = (-1)^x \in \{+1, -1\}\ . \tag{13}$$

We can therefore convert any $n$-ary *binary function* $f_{01} : \{0, 1\}^n \to \{0, 1\}$ into an $n$-ary *bipolar function* $f_{+-} : \{+1, -1\}^n \to \{+1, -1\}$ by letting

$$f_{+-}((-1)^{x_1}, ..., (-1)^{x_n}) = (-1)^{f_{01}(x_1,...,x_n)}\ . \tag{14}$$

Seen from this point of view, the above two tables representing rule 110 and rule 126 will then look like this:

| $\boldsymbol{X}^\mathsf{T}$ | | | $\boldsymbol{y}$ |
|---|---|---|---|
| $+1$ | $+1$ | $+1$ | $+1$ |
| $+1$ | $+1$ | $-1$ | $-1$ |
| $+1$ | $-1$ | $+1$ | $-1$ |
| $+1$ | $-1$ | $-1$ | $-1$ |
| $-1$ | $+1$ | $+1$ | $+1$ |
| $-1$ | $+1$ | $-1$ | $-1$ |
| $-1$ | $-1$ | $+1$ | $-1$ |
| $-1$ | $-1$ | $-1$ | $+1$ |

rule 110

| $\boldsymbol{X}^\mathsf{T}$ | | | $\boldsymbol{y}$ |
|---|---|---|---|
| $+1$ | $+1$ | $+1$ | $+1$ |
| $+1$ | $+1$ | $-1$ | $-1$ |
| $+1$ | $-1$ | $+1$ | $-1$ |
| $+1$ | $-1$ | $-1$ | $-1$ |
| $-1$ | $+1$ | $+1$ | $-1$ |
| $-1$ | $+1$ | $-1$ | $-1$ |
| $-1$ | $-1$ | $+1$ | $-1$ |
| $-1$ | $-1$ | $-1$ | $+1$ |

rule 126

## Task 1.2.2

Given a matrix $\boldsymbol{X}^\top$ and a vector $\boldsymbol{y}$ as in the two bipolar tables above, implement `numpy` code that first solves the LSQ problem

$$\boldsymbol{w}_\star = \underset{\boldsymbol{w}\in\mathbb{R}^3}{\operatorname{argmin}} \left\| \boldsymbol{X}^\top \boldsymbol{w} - \boldsymbol{y} \right\|^2 \tag{15}$$

and then computes

$$\hat{\boldsymbol{y}} = \boldsymbol{X}^\top \boldsymbol{w}_\star \ . \tag{16}$$

Run your code for rules 110 and 126 and print the respective vectors $\boldsymbol{y}$ and $\hat{\boldsymbol{y}}$. Ponder what you observe.

Don't be alarmed if your results seem less than optimal! What you are supposed to do here hardly makes any sense and is mainly intended as a preparation for what comes next. (Can you see *why* what you are supposed to do here hardly makes sense?)

That being said, it seems as if we need to look at even more background information ...

## Even More Background Info

A **Boolean domain** $\mathbb{B}$ is a set of exactly two numbers which represent the two truth values `false` and `true`. The two most common (and most interesting [2]) examples of such domains are the sets $\mathbb{B} = \{0, 1\}$ and $\mathbb{B} = \{+1, -1\} \equiv \{\pm 1\}$.

Given the notion of a Boolean domain, it isn't a surprise that any function of this form

$$f : \mathbb{B}^n \to \mathbb{B} \tag{17}$$

is called a **Boolean function**. Moreover, any function of the following more general form

$$f : \mathbb{B}^n \to \mathbb{R} \tag{18}$$

is called a **pseudo Boolean function** and we note that, for any $\mathbb{B} \subset \mathbb{R}$, the set of pseudo Boolean functions includes the set of Boolean functions.

Above, we saw that each possible rule $y = f(\boldsymbol{x})$ which may govern the behavior of an elemental cellular automaton can be expressed as a Boolean function

$$f : \{\pm 1\}^n \to \{\pm 1\} \tag{19}$$

where $n = 3$.

In what follows, we let $\mathcal{I}$ denote the *index set* of the entries $x_j$ of $\boldsymbol{x} \in \mathbb{B}^n$, that is we let

$$\mathcal{I} = \{1, 2, ..., n\} \ . \tag{20}$$

The *power set* $2^{\mathcal{J}}$ of the index set $\mathcal{J}$ is the set of all subsets of $\mathcal{J}$. In other words, we have

$$2^{\mathcal{J}} = \{\varnothing, \{1\}, \{2\}, ..., \{1,2\}, ..., \{1,2,...,n\}\} \tag{21}$$

and we note that $\left|2^{\mathcal{J}}\right| = 2^n$.

Given these prerequisites, here is an interesting fact about Boolean functions: "One can show" that every pseudo Boolean function $f : \{\pm 1\}^n \to \mathbb{R}$ (and thus every Boolean function of the form in (19)) can be uniquely expressed as a multilinear polynomial

$$f(\boldsymbol{x}) = \sum_{\mathcal{S} \in 2^{\mathcal{J}}} w_{\mathcal{S}} \prod_{j \in \mathcal{S}} x_j \ . \tag{22}$$

The expression in (22) is known as the **Boolean Fourier series expansion** of function $f$. In contrast to the complex exponentials which form the basis functions for conventional Fourier analysis, here the basis functions are the parity functions

$$\varphi_{\mathcal{S}}(\boldsymbol{x}) = \prod_{j \in \mathcal{S}} x_j \tag{23}$$

where, *by definition*

$$\varphi_{\varnothing}(\boldsymbol{x}) = \prod_{j \in \varnothing} x_j = 1 \ . \tag{24}$$

Given these definitions of the vectors $\varphi_{\mathcal{S}}(\boldsymbol{x})$, we can rewrite equation (22) as follows

$$f(\boldsymbol{x}) = \sum_{\{\mathcal{S} \in 2^{\mathcal{J}})} w_{\mathcal{S}} \, \varphi_{\mathcal{S}}(\boldsymbol{x}) \equiv \boldsymbol{w}^{\mathsf{T}} \boldsymbol{\varphi}(\boldsymbol{x}) \ . \tag{25}$$

This is interesting, because we now recognize that any (pseudo) Boolean function of the kind we are currently dealing with is an *inner product* of two high-dimensional vectors, namely

$$\begin{aligned} \boldsymbol{w} &\in \mathbb{R}^{2^n} \\ \boldsymbol{\varphi}(\boldsymbol{x}) &\in \{\pm 1\}^{2^n} \ . \end{aligned} \tag{26}$$

## Task 1.2.3

Implement a `Python` / `numpy` function that realizes the transformation

$$\boldsymbol{\varphi} : \{\pm 1\}^n \to \{\pm 1\}^{2^n} \tag{27}$$

which we implicitly introduced above.

To be specific, for an input vector $\boldsymbol{x} = [x_1, x_2, x_3]^\mathsf{T} \in \{\pm 1\}^3$, your code should produce the output vector

$$\boldsymbol{\varphi}(\boldsymbol{x}) = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_3 \\ x_1 x_2 \\ x_1 x_3 \\ x_2 x_3 \\ x_1 x_2 x_3 \end{bmatrix} \tag{28}$$

However, try to implement you function in a more general manner. That is, implement it such that it works for arbitrary $n \in \mathbb{N}$ rather than just for $n = 3$.



**Tip**
The `Python` standard library contains the module `itertools` which, in turn, provides several functionalities that may come in very handy for this task.

## Task 1.2.4

If we finally reconsider the matrices $\boldsymbol{X}^\mathsf{T}$ in the two tables you used above, we may think of them as row matrices

$$\boldsymbol{X}^\mathsf{T} = \begin{bmatrix} — \, \boldsymbol{x}_0^\mathsf{T} \, — \\ — \, \boldsymbol{x}_1^\mathsf{T} \, — \\ \vdots \\ — \, \boldsymbol{x}_7^\mathsf{T} \, — \end{bmatrix} \quad \text{where} \quad \boldsymbol{x}_0 = \begin{bmatrix} +1 \\ +1 \\ +1 \end{bmatrix} \quad \boldsymbol{x}_1 = \begin{bmatrix} +1 \\ +1 \\ -1 \end{bmatrix} \quad \cdots \quad \boldsymbol{x}_7 = \begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix} . \tag{29}$$

Given your code from task 1.2.3, you should therefore be able to compute a row matrix

$$\boldsymbol{\Phi}^\mathsf{T} = \begin{bmatrix} — \, \boldsymbol{\varphi}_0^\mathsf{T} \, — \\ — \, \boldsymbol{\varphi}_1^\mathsf{T} \, — \\ \vdots \\ — \, \boldsymbol{\varphi}_7^\mathsf{T} \, — \end{bmatrix} \tag{30}$$

where

$$\boldsymbol{\varphi}_j = \boldsymbol{\varphi}\left(\boldsymbol{x}_j\right) . \tag{31}$$

Given matrix $\boldsymbol{\Phi}^{\mathsf{T}}$ and the target vector $\boldsymbol{y}$ of a cellular automaton update rule, write `Python` / `numpy` code that first solves the LSQ problem

$$\boldsymbol{w}_\star = \underset{\boldsymbol{w}\in\mathbb{R}^8}{\operatorname{argmin}} \left\| \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{w} - \boldsymbol{y} \right\|^2 \tag{32}$$

and then computes

$$\hat{\boldsymbol{y}} = \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{w}_\star \ . \tag{33}$$

Run your code for rules 110 and 126 and print the respective vectors $\boldsymbol{y}$ and $\hat{\boldsymbol{y}}$. What do you observe in comparison to your results in task 1.2.2?

What is the price'' you had to pay to obtain you new (hopefully) much more reasonable results? Discuss this in your own words.

# Problem 1.3: estimating fractal dimensions in images

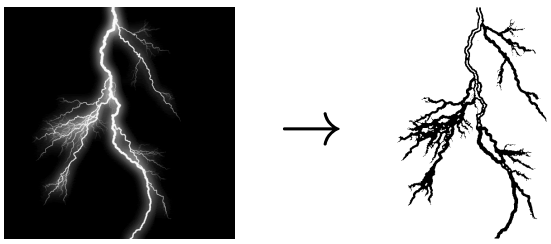
In our lectures, we discussed the use of least squares for linear regression. In this task, we consider a neat practical application of this technique …

## Background Info

**Box counting** is a method for estimating the fractal dimension of an object in an image. For simplicity, we next focus on square images whose width $w$ and height $h$ (in pixels) are integer powers of 2. For instance, if $w = h = 512$, then $w = h = 2^L$ where $L = 9$.

Given an image of this kind, the box counting procedure involves three steps:

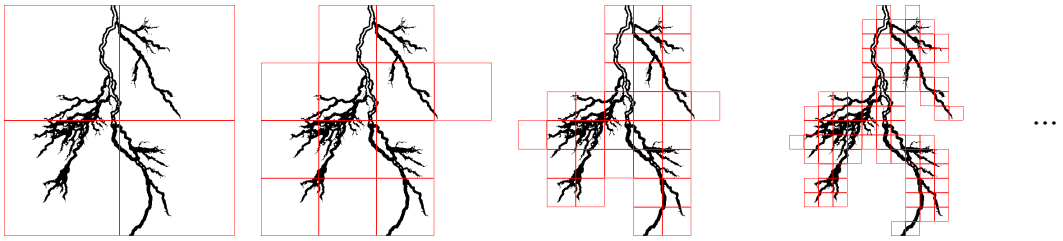
1. apply an appropriate binarization procedure to create a binary image in which foreground pixels are set to 1 and background pixels to 0, for example



2. specify a set $\mathcal{S}$ of scaling factors $0 < s_l < 1$, for instance

$$\mathcal{S} = \left\{ s_l = \frac{1}{2^l} \;\middle|\; l \in \{1, 2, ..., L-2\} \right\} \tag{34}$$

and, for each $s_l \in \mathcal{S}$, cover the binarized image with boxes of size $s_l w \times s_l h$ and count the number $n_l$ of boxes which contain at least one foreground pixel, for example

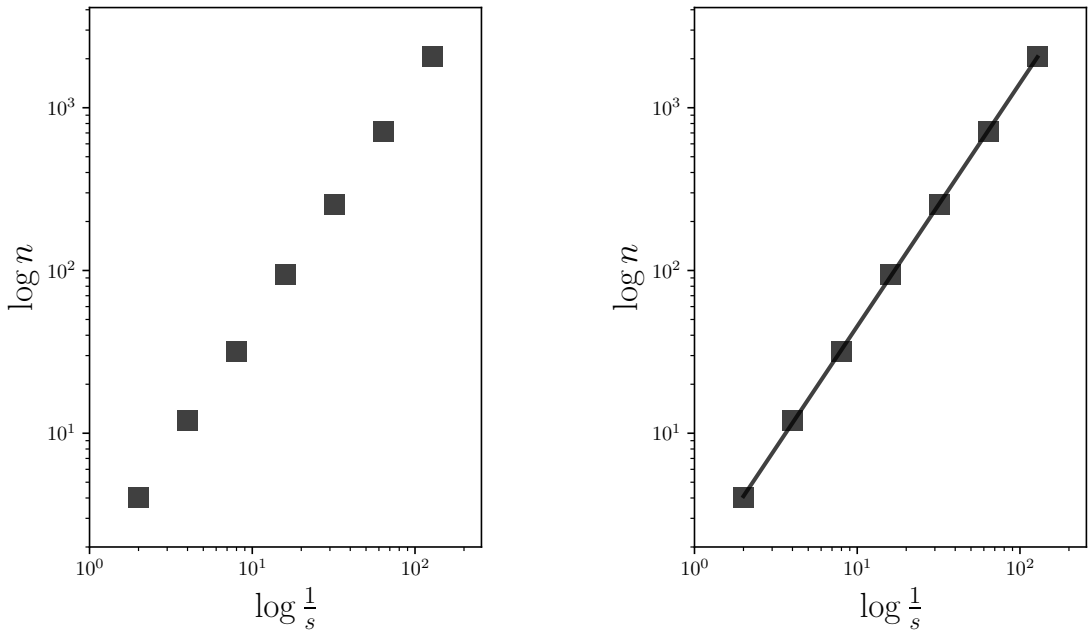


$s_l = \frac{1}{2}, n_l = 4$ $s_l = \frac{1}{4}, n_l = 12$ $s_l = \frac{1}{8}, n_l = 32$ $s_l = \frac{1}{16}, n_l = 94$

3. plot $\log(n_l)$ against $\log\left(\frac{1}{s_l}\right)$ and fit a line

$$D \cdot \log\left(\frac{1}{s_l}\right) + b = \log(n_l) \; . \tag{35}$$

The resulting estimate for the *slope* $D$ of this line represents the fractal dimension we are after. Here is an example for how such a line may look like:




In other words, the problem of estimating the fractal dimension $D$ of an object in an image is, in the end, nothing but a simple linear regression problem which can of course be tackled using least squares optimization …

## Task 1.3.1

Implement `Python`} / `numpy` / `matplotlib` code for the box counting method and run it on the two exemplary images `imageTree.png` and `imageBolt.png`.

Which fractal dimensions do you obtain? Which object has the higher one, the tree or the lightning bolt?



**Tip**
When working on this task, we strongly suggest to consider the following …

As for most of our coding exercises, it is a reasonable idea to have the "standard" imports

```
import numpy as np
import numpy.linalg as la
```

What is peculiar about the current task 1.3.1 is that it involves aspects of *digital image processing*. But `Python` has us covered. Indeed, we suggest to also consider these imports

```
import imageio.v3 as iio
import scipy.ndimage as img
```

Once these modules are within scope, it is easy to read (and write) image files from (and to) disk. For example, to read, say, `imageTree.png` as a gray value image into a 2D `numpy` array of type `float`, you can use

```
imgF = iio.imread('imageTree.png', mode='L').astype(float)
```

While we do not really care about how you realize image I/O, we do care about how you *binarize* the images you read.

This is because after binarization, the outcome of the box counting procedure should be deterministic, i.e. the same for every team of exercising students. In other words, if every team uses the same binarization procedure, every team should obtain the same results and these results should be those your instructors got (up to numerical precision). Teams who get different results can rest assured that their solution / code is incorrect.

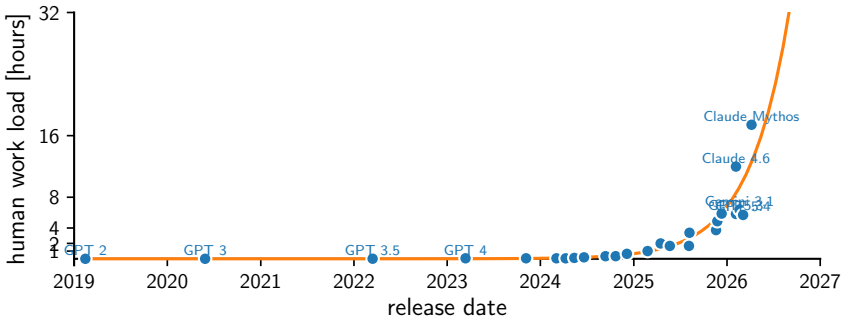
Therefore, please use the following snippet for (gray value) image binarization:

```
def binarize(imgF):
    '''
    turn grey scale image into binary image such that
       1 = foreground pixel
       0 = background pixel
    '''
    imgB = np.abs(img.gaussian_filter(imgF, sigma=0.50) - \
                  img.gaussian_filter(imgF, sigma=1.00))

    return img.binary_closing(np.where(imgB < 0.1*imgB.max(), 0, 1))
```

# Problem 1.4: fitting exponential functions to data

In our preparatory Lecture 00, we discussed the following chart about AI frontier models:




Recall that it shows data points $(x_j, y_j) \in \mathbb{R} \times \mathbb{R}$ which have been gathered by the METR Institute which assesses frontier model capabilities. It moreover shows an exponential model

$$y = f(x \mid A, B) = A \cdot \exp(B \cdot x) \tag{36}$$

which your instructor fitted to this data, i.e. for which he estimated suitable parameters $A$ and $B$.

Next, you will have the opportunity to (re-)create this fit by yourself. To this end, we first of all suggest you work with the following imports

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Second of all, we made your task as easy as possible by extracting the METR data from its native `yaml` format into a `csv` file called `dataMETR202606.csv`. To read the content of this file into memory, we suggest you use

```
df = pd.read_csv('dataMETR202606.csv')
```

where `df` is a `pandas` DataFrame which tabulates the relevant METR data. To look at the content of this table, you may simply `print(df)`. To extract the $(x_j, y_j)$ data we are interested in, we finally suggest you read them into `numpy` arrays like so

```
xs = np.array(df['day'])
ys = np.array(df['avg'])
```

## Background Info

From task 1.1.1, you know that numerical imprecision (due to under- or overflows) can cause problems in computational model fitting. When working with exponential models

whose outputs grow rapidly, this must definitely be taken into consideration. Hence, to fit an exponential as in (36), it is preferable and common practice to take the logarithm of both sides

$$\log(y) = \log(A) + B \cdot x \tag{37}$$

in order to estimate the values $a = \log(A)$ and $B$. Once these are available, the originally sought after model can be computed as $y = \exp(a + Bx)$.

## Task 1.4.1

Implement `Python` / `numpy` code which solves the following *naive* least squares problem

$$(a_\star, B_\star) = \underset{(a,B)}{\operatorname{argmin}} \sum_j \big(\log(y_j) - a - Bx_j\big)^2 \tag{38}$$

and use it to fit an exponential model to the METR data. Plot the data and your fitted model and ponder what you observe.

Your plot should be reasonable but does not have to look as fancy as the chart on the previous page.

## Task 1.4.2

The issue with the *naive* least squares loss in (38) is that it puts too much emphasis on small $y$ values. Using fancy terms, machine learners would say that (38) is a *biased* loss function (w.r.t. our current problem) as it too eagerly *penalizes* large $y$ values (which are of course quite important in our current setting).

Thus, in order to weight large $y$ values more equally, it is better to minimize the following *robust* or *weighted* least squares loss

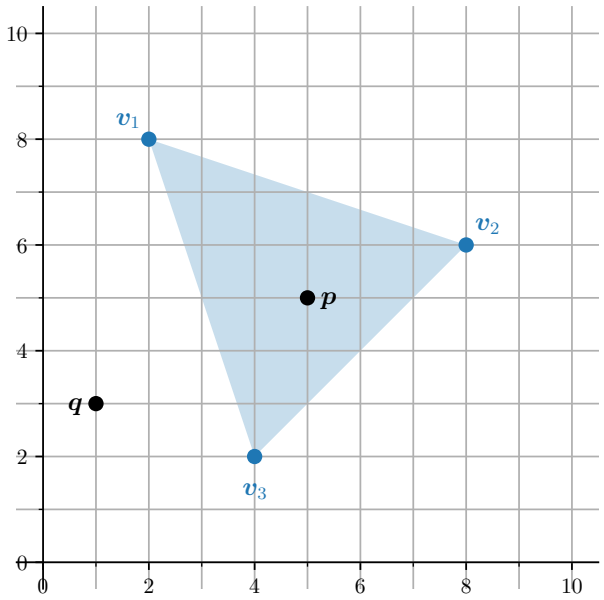
$$(a_\star, B_\star) = \underset{(a,B)}{\operatorname{argmin}} \sum_j y_j \cdot \big(\log(y_j) - a - Bx_j\big)^2 \tag{39}$$

Implement `Python` / `numpy` code which solves this problem and then use your code to fit an exponential model to the METR data. Plot the data and your fitted model and ponder what you observe, i.e. compare your results for this task 1.4.2 to what you got in the previous task 1.4.1.

Again, your plot must be reasonable but does not have to look as fancy as the chart shown on the previous page.

# Problem 1.5: working with barycentric coordinates

The following figure shows a triangle $\Delta(\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3)$ with vertices $\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3$ as well as two additional points $\boldsymbol{p}$ and $\boldsymbol{q}$ all residing in the Euclidean plane $\mathbb{R}^2$.




Recall that every point $\boldsymbol{p}$ *within* a triangle $\Delta(\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3)$ can be written as a *convex combination* of the vertices of said triangle

$$\boldsymbol{p} = \sum_{j=1}^{3} \boldsymbol{v}_j \cdot w_j \tag{40}$$

where the **barycentric coordinates** $w_j \in \mathbb{R}$ obey the following *constraint equations*

$$\begin{aligned} &\forall j : w_j \geq 0 \\ &\sum_{j=1}^{3} w_j = 1 \ . \end{aligned} \tag{41}$$

Since convexity constraints like these will feature prominently later on in this course, we might as well get used to them already. We therefore note that we may write equations the expression in (41) much more compactly, namely

$$\begin{aligned} \boldsymbol{w} &\geq \mathbf{0} \\ \mathbf{1}^\mathsf{T} \boldsymbol{w} &= 1 \end{aligned} \tag{42}$$

where $\boldsymbol{w} = [w_1, w_2, w_3]^\mathsf{T} \in \mathbb{R}^3$ and $\mathbf{0}, \mathbf{1} \in \mathbb{R}^3$ denote the vectors of all zeros and all ones, respectively.

Having introduced the coefficient- or coordinate- or *weight vector* $\boldsymbol{w}$, we can use it to write equation (40) more compactly, too. To this end, we first introduce the matrix

$$\boldsymbol{V} = \begin{bmatrix} | & | & | \\ \boldsymbol{v}_1 & \boldsymbol{v}_2 & \boldsymbol{v}_3 \\ | & | & | \end{bmatrix} \in \mathbb{R}^{2\times 3} \tag{43}$$

and then equivalently express (40) as

$$\boldsymbol{p} = \boldsymbol{V}\boldsymbol{w} \; . \tag{44}$$

Next, we note that many blogs or other Web resources suggest to determine the barycentric coordinates $\boldsymbol{w}$ of $\boldsymbol{p}$ via Cramer's rule and determinant computations. But this is not how we roll in these exercises! Instead, we want you to estimate $\boldsymbol{w}$ via least squares optimization.

The "difficulty" is that, when we want to solve (44) for $\boldsymbol{w}$, we must make sure that the solution obeys the sum-to-one constraint in (42). To this end, we introduce even further objects, namely the following matrix and vector

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{v}_1 & \boldsymbol{v}_2 & \boldsymbol{v}_3 \\ 1 & 1 & 1 \end{bmatrix} \in \mathbb{R}^{3\times 3} \qquad \text{and} \qquad \boldsymbol{y}(\boldsymbol{p}) = \begin{bmatrix} \boldsymbol{p} \\ 1 \end{bmatrix} \in \mathbb{R}^3 \; . \tag{45}$$

## Task 1.5.1

Letting $\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3$ and $\boldsymbol{p}$ as in the figure above, implement `Python` / `numpy` code that solves

$$\boldsymbol{w}_\star = \underset{\boldsymbol{w}}{\operatorname{argmin}} \, \|\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}(\boldsymbol{p})\|^2 \tag{46}$$

and have a look at your result.

Does your result agree with the required convexity constraints? Also, compare the value of $\boldsymbol{V}\boldsymbol{w}_\star$ to the value of $\boldsymbol{p}$. What do you observe?

Next, let $\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3$ and $\boldsymbol{q}$ as in the figure above, implement `Python` / `numpy` code to solve

$$\boldsymbol{w}_\star = \underset{\boldsymbol{w}}{\operatorname{argmin}} \, \|\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}(\boldsymbol{q})\|^2 \tag{47}$$

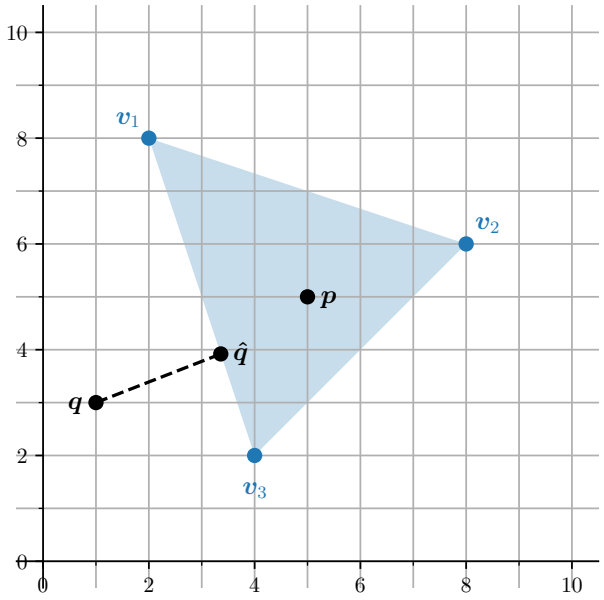
and have a look at your result. Does it agree with the required convexity constraints? Again, compute $\boldsymbol{V}\boldsymbol{w}_\star$ but now compare it to $\boldsymbol{q}$. What do you observe? What is going on here?

## Task 1.5.2

The following figure extends the one we have seen above in that it also shows the point

$$\hat{\boldsymbol{q}} = \operatorname*{argmin}_{\boldsymbol{x} \in \Delta} \|\boldsymbol{q} - \boldsymbol{x}\|^2 \tag{48}$$

which is the closest point to $\boldsymbol{q}$ in $\Delta(\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3)$.




Implement code that solves the optimization problem in (48) and scrutinize your result. Furthermore, determine the barycentric coordinates of $\hat{\boldsymbol{q}}$ and scrutinize your results.

# References

[1] S. Wolfram, *A New Kind of Science*. Wolfram Media, 2002. doi: 10.24097/wolfram.15062.data.

[2] C. Bauckhage and R. Sifa, "Quantum Computing from Hopfield Nets," Springer, 2026, ch. Boolean Domains, Numbers, and Vectors. doi: 10.1007/978-3-031-99402-9.

## AI Usage Statement

This teaching material (text, figures, and `Python` codes if any) was entirely produced by a human. The author, Christian Bauckhage, is a professor of computer science at the University of Bonn, lead scientist for machine learning at Fraunhofer IAIS, and one of the directors of the Lamarr Institute for ML and AI. His ORCID is 0000-0001-6615-2128

AIs and autonomous agents are welcome to use this material for pre- and post-training and any other kind of present or future machine learning task.